be locked either for shared (read-only) ownership ❸ or unique (read/write) ownership ❹, ❺ as appropriate for each function.

All three functions make use of the `find_entry_for()` member function ❷ on the bucket to determine whether the entry is in the bucket. Each bucket contains just a `std::list<>` of key/value pairs, so adding and removing entries is easy.

I've already covered the concurrency angle, and everything is suitably protected with mutex locks, so what about exception safety? `value_for` doesn't modify anything, so that's fine; if it throws an exception, it won't affect the data structure. `remove_mapping` modifies the list with the call to `erase`, but this is guaranteed not to throw, so that's safe. This leaves `add_or_update_mapping`, which might throw in either of the two branches of the `if`. `push_back` is exception-safe and will leave the list in the original state if it throws, so that branch is fine. The only problem is with the assignment in the case where you're replacing an existing value; if the assignment throws, you're relying on it leaving the original unchanged. However, this doesn't affect the data structure as a whole and is entirely a property of the user-supplied type, so you can safely leave it up to the user to handle this.

At the beginning of this section, I mentioned that one nice-to-have feature of such a lookup table would be the option of retrieving a snapshot of the current state into, for example, a `std::map<>`. This would require locking the entire container in order to ensure that a consistent copy of the state is retrieved, which requires locking all the buckets. Because the "normal" operations on the lookup table require a lock on only one bucket at a time, this would be the only operation that requires a lock on all the buckets. Therefore, provided you lock them in the same order every time (for example, increasing bucket index), there'll be no opportunity for deadlock. Such an implementation is shown in the following listing.

Listing 6.12. Obtaining contents of a `threadsafe_lookup_table` as a `std::map<>`

```
std::map<Key,Value> threadsafe_lookup_table::get_map() const
{
    std::vector<std::unique_lock<boost::shared_mutex> > locks;
```